CHEAT SHEET – APARTMENT HUNTER

Fiche de compréhension du projet, de A à Z. Objectif : que n'importe qui dans l'équipe (Romain, Yannis, Lisa) puisse comprendre et expliquer **tout** le projet, même sans avoir écrit chaque ligne.

> Tous les chiffres de cette fiche viennent du notebook exécuté (`apartment_hunter.ipynb`) et du code réel (`app.py` , `Dockerfile` , `requirements.txt`).

---

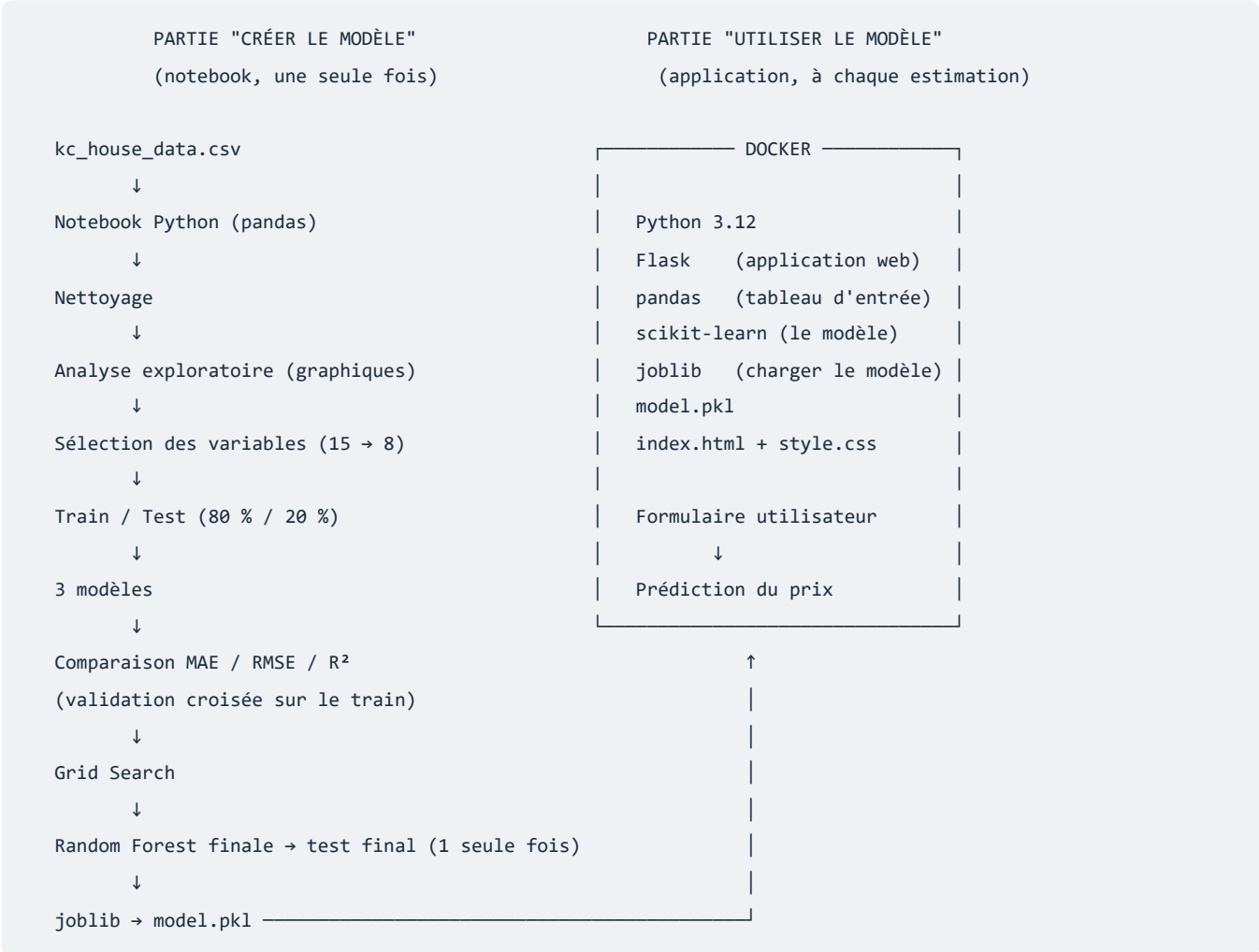
0. LA QUESTION DU PROJET

> **Peut-on estimer le prix d'une maison à partir de ses caractéristiques ?**

Réponse : oui. Notre modèle final (une Random Forest) explique environ **87,7 % de la variation des prix** sur des maisons qu'il n'a jamais vues, avec une erreur moyenne d'environ **71 700 $**.

---

1. ARCHITECTURE GLOBALE DU PROJET



En clair :

- **Le notebook** sert à **créer et entraîner** le modèle. On le lance une fois, il produit `model.pkl`.
- **Flask** sert à **utiliser** le modèle depuis une page web : l'utilisateur remplit un formulaire, Flask donne les valeurs au modèle et affiche le prix.
- **Docker** sert à **emballer** l'application et toutes ses dépendances, pour qu'elle fonctionne de la même manière sur n'importe quel ordinateur.

---

2. POURQUOI ON A FAIT ÇA ? (fiche de révision n°1)

Choix	Pourquoi
Dataset King County (pas Madrid)	Moins de colonnes, pas de texte libre, variables bien documentées : plus simple à nettoyer et à expliquer
Supprimer `id`	C'est un identifiant : le numéro d'une vente n'explique pas son prix
Supprimer `date`	Les ventes ne couvrent qu'environ 13 mois (2 mai 2014 → 27 mai 2015). On n'a pas étudié l'effet du temps pour garder le projet simple
Supprimer `zipcode`	C'est un code, pas une quantité (98178 n'est pas « plus » que 98125). `lat` et `long` donnent déjà la position
Supprimer `sqft_above` et `sqft_basement`	Leur somme est exactement égale à `sqft_living` pour toutes les maisons : information en double
Supprimer 17 lignes	Erreurs évidentes : 1 maison à 33 chambres pour 1 620 pieds², 16 maisons à 0 chambre ou 0 salle de bain
Garder les maisons très chères	Valeurs extrêmes mais plausibles (grandes maisons de luxe). 1 145 maisons (5,3 %) au-dessus de la limite IQR : ce ne sont pas des erreurs
Pas de normalisation	Inutile pour les arbres, et ne change pas les prédictions de la régression linéaire. L'application reste simple
Garder 8 variables sur 15	Même performance (R^2 0,855 contre 0,856) avec un modèle plus simple et un formulaire plus court
Comparer en validation croisée	Pour choisir le modèle **sans regarder le test**
Random Forest	Meilleur résultat sur les 3 mesures
Grid Search	Tester automatiquement plusieurs réglages de la Random Forest
Test utilisé une seule fois	Pour avoir un score final honnête, jamais utilisé pour faire un choix
joblib + `model.pkl`	Sauvegarder le modèle entraîné pour ne pas le réentraîner à chaque lancement
`compress=3`	Fichier de 26,3 Mo au lieu de beaucoup plus : reste sous la limite de 100 Mo de GitHub
Flask	Rendre le modèle utilisable depuis une page web
Docker	Rendre l'environnement reproductible : même Python, mêmes librairies partout
matplotlib + `dashboard.html`	On n'avait pas Power BI : analyse refaite en Python, version consultable en HTML

---

3. À QUOI SERT CHAQUE FICHIER ?

Fichier	À quoi il sert	Quand il intervient
`data/kc_house_data.csv`	Les données brutes : 21 613 ventes de maisons, 21 colonnes	Au tout début, chargé par le notebook
`apartment_hunter.ipynb`	Toute l'analyse : nettoyage, graphiques, sélection des variables, entraînement et évaluation des modèles	Phase de création du modèle
`images/`	Les 9 graphiques générés par le notebook	Créés par le notebook, utilisés dans le README et le dashboard
`dashboard.html`	Version consultable de l'analyse exploratoire (remplace Power BI)	Pour présenter l'analyse, à ouvrir dans un navigateur
`model.pkl`	La Random Forest déjà entraînée et sauvegardée	Créé à la fin du notebook, chargé par `app.py`
`app.py`	L'application Flask : récupère les valeurs du formulaire et demande au modèle de prédire un prix	Quand on lance l'application
`templates/index.html`	La page visible par l'utilisateur (le formulaire et le résultat)	À chaque affichage de la page
`static/style.css`	La mise en forme de la page (carte centrée, bouton bleu, encadré vert)	À chaque affichage de la page
`requirements.txt`	La liste des librairies de l'application (avec leurs versions)	Installation locale ou construction Docker
`Dockerfile`	La recette pour construire l'environnement Docker	`docker build`
`README.md`	La documentation du projet sur GitHub	Lecture du dépôt
`.gitignore`	Dit à Git d'ignorer les fichiers temporaires (`__pycache__`, checkpoints)	Quand on fait un commit

---

4. QUE SE PASSE-T-IL QUAND L'UTILISATEUR CLIQUE SUR « ESTIMER » ?

Avant même le clic : au démarrage, `app.py` charge `model.pkl` **une seule fois** avec `joblib.load`. Le modèle reste ensuite en mémoire.

1. L'utilisateur entre les 8 caractéristiques de la maison dans le formulaire.
2. Le formulaire HTML envoie les valeurs à Flask (requête **POST** sur la page `/`).
3. `app.py` récupère les 8 valeurs avec `request.form` et les convertit en nombres (`float`).
4. Flask crée un **DataFrame pandas** d'une ligne, avec les colonnes **dans le même ordre** que pendant l'entraînement.
5. La Random Forest (déjà chargée depuis `model.pkl`) reçoit ces 8 variables.
6. Chacun des 200 arbres donne un prix, le modèle fait la **moyenne** : c'est l'estimation.
7. Le prix est mis en forme (`375 210`) et Flask renvoie la page HTML avec le résultat dans l'encadré vert.

```
Utilisateur → formulaire HTML → Flask (app.py) → DataFrame pandas → Random Forest (model.pkl) → prix estimé → pag
```

En une phrase à l'oral :

« Quand on clique sur Estimer, Flask récupère les 8 valeurs du formulaire, les met dans un tableau pandas, les donne à la Random Forest déjà chargée, et renvoie le prix estimé dans la page. »

Exemple à montrer en démo : 3 chambres, 1 510 pieds², terrain 6 083 pieds², pas de vue sur l'eau, grade 6, jamais rénovée, latitude 47.6966, longitude -122.324 → **375 210 $** (maison réellement vendue 415 000 $).

---

5. LE NOTEBOOK EXPLIQUÉ COMME SI J'AVAIS 12 ANS

1. Chargement des données

- **C'est quoi ?** Ouvrir le fichier CSV dans Python, sous forme de tableau.
- **Pourquoi ?** On ne peut rien faire sans les données.
- **Ici :** `pd.read_csv("data/kc_house_data.csv")` puis `head()`, `shape`, `info()`, `describe()`. Résultat : 21 613 lignes, 21 colonnes.
- **À retenir :** « On commence toujours par regarder ce qu'il y a dans les données avant de les modifier. »

2. Vérification des valeurs manquantes

- **C'est quoi ?** Chercher les cases vides.
- **Pourquoi ?** Beaucoup de modèles plantent ou se trompent s'il y a des trous.
- **Ici :** `df.isna().sum()` → **0** valeur manquante dans toutes les colonnes.
- **À retenir :** « Il n'y avait aucune valeur manquante, donc pas besoin de supprimer ni de remplacer des valeurs. »

3. Vérification des doublons

- **C'est quoi ?** Chercher les lignes en double.
- **Pourquoi ?** Une même ligne comptée deux fois fausserait l'apprentissage.
- **Ici :** 0 ligne en double. 177 `id` apparaissent deux fois, mais toujours avec une date différente : c'est la même maison vendue deux fois.
- **À retenir :** « On a gardé ces ventes répétées, car ce sont de vraies ventes, pas des erreurs. »

4. Suppression des colonnes inutiles

- **C'est quoi ?** Enlever les colonnes qui n'aident pas à prédire le prix.
- **Pourquoi ?** Un modèle plus simple, et moins de champs dans l'application.
- **Ici :** on enlève `id`, `date`, `zipcode`, `sqft_above`, `sqft_basement`. On passe de 21 à 16 colonnes.
- **À retenir :** « On a enlevé les identifiants, ce qu'on ne voulait pas traiter (la date), et les informations en double. »

5. Suppression des valeurs incohérentes

- **C'est quoi ?** Enlever les lignes qui sont clairement des erreurs de saisie.
- **Pourquoi ?** Le modèle apprendrait à partir de fausses informations.
- **Ici :** 1 maison à 33 chambres pour 1 620 pieds² (sûrement une faute de frappe pour 3), et 16 maisons sans chambre ou sans salle de bain. Total : 17 lignes, on garde **21 596** maisons.
- **À retenir :** « On a supprimé les lignes **impossibles**, pas les lignes **extrêmes**. Les maisons de luxe sont restées. »

6. Analyse exploratoire

- **C'est quoi ?** Faire des graphiques pour comprendre les données.
- **Pourquoi ?** Voir quelles variables semblent influencer le prix avant de faire des modèles.

- **Ici :** 7 graphiques (distribution des prix, surface, grade, chambres, vue, carte, corrélations). On voit que le prix dépend surtout de la **surface**, de la **qualité** et de l'**emplacement**.
- **À retenir :** « Le sujet demandait Power BI, on l'a remplacé par matplotlib et un `dashboard.html`. »

7. Train / Test

- **C'est quoi ?** Couper les maisons en deux groupes.
- **Pourquoi ?** On ne veut pas vérifier le modèle avec les maisons qu'il a utilisées pour apprendre.
- **Ici :** 80 % pour apprendre (**17 276** maisons) et 20 % pour le test final (**4 320** maisons), avec `random_state=42`. Tous les choix sont faits sur le train ; le test ne sert qu'**une fois**, à la fin.
- **À retenir :** « On garde des maisons que le modèle n'a jamais vues pour vérifier s'il sait généraliser. »

8. Corrélation

- **C'est quoi ?** Un nombre entre -1 et 1 qui dit si deux variables montent ensemble « en ligne droite ».
- **Pourquoi ?** Première idée simple des variables liées au prix.
- **Ici :** `sqft_living` 0,70, `grade` 0,67, `sqft_living15` 0,59, `bathrooms` 0,53… mais `long` seulement 0,017 alors que la carte montre que l'emplacement compte.
- **À retenir :** « La corrélation ne voit que les liens en ligne droite, et elle ne voit pas quand deux variables disent la même chose. »

9. SequentialFeatureSelector (sélection forward)

- **C'est quoi ?** Une méthode qui part de zéro variable et ajoute, à chaque étape, celle qui améliore le plus le modèle.
- **Pourquoi ?** Elle regarde ce qu'une variable apporte **en plus** des autres, contrairement à la corrélation.
- **Ici :** avec un arbre de décision limité à 10 niveaux, `n_features_to_select=8`, `cv=3`, sur le train uniquement. Elle garde `lat` et `long`, et laisse de côté `bathrooms` (déjà dans la surface).
- **À retenir :** « On commence avec aucune variable puis on ajoute progressivement celles qui améliorent le modèle. »

10. Régression linéaire

- **C'est quoi ?** Une formule : prix ≈ a × surface + b × grade + … + constante.
- **Pourquoi ?** C'est le modèle de base le plus simple.
- **Ici :** R^2 de **0,652** en validation croisée. Le moins bon.
- **À retenir :** « Elle ne voit que des relations en ligne droite, alors que l'effet de l'emplacement n'est pas en ligne droite. »

11. Arbre de décision

- **C'est quoi ?** Une suite de questions oui/non (« surface > 2 500 ? », « grade > 7 ? ») qui range les maisons en groupes de prix proches.
- **Pourquoi ?** Il peut représenter des relations qui ne sont pas en ligne droite.
- **Ici :** R^2 de **0,740** en validation croisée, mais **0,999** sur les maisons apprises : il apprend par cœur.
- **À retenir :** « Un arbre seul, sans limite, fait du surapprentissage. »

12. Random Forest (forêt aléatoire)

- **C'est quoi ?** Beaucoup d'arbres, chacun entraîné sur un tirage au hasard des maisons. On fait la **moyenne** de leurs prédictions.
- **Pourquoi ?** Les erreurs des différents arbres se compensent.
- **Ici :** R^2 de **0,855** en validation croisée (par défaut), **0,856** après Grid Search. La meilleure.
- **À retenir :** « Plein d'arbres un peu différents, et on fait la moyenne : c'est plus fiable qu'un seul arbre. »

13. Validation croisée

- **C'est quoi ?** On coupe le train en 3 parts, on entraîne sur 2 et on mesure sur la 3e, trois fois, puis on fait la moyenne.
- **Pourquoi ?** Comparer des modèles et des réglages **sans toucher au jeu de test**.
- **Ici :** `cv=3` partout (sélection forward, comparaison des modèles, Grid Search).
- **À retenir :** « La validation croisée sert à choisir. Le test sert à vérifier, une seule fois, à la fin. »

14. MAE

- **C'est quoi ?** L'erreur moyenne, en dollars.
- **Pourquoi ?** C'est la mesure la plus facile à comprendre.
- **Ici : 71 718 $** sur le test final (73 764 $ en validation croisée).
- **À retenir :** « En moyenne, le modèle se trompe d'environ 72 000 $. »

15. RMSE

- **C'est quoi ?** Une erreur en dollars où les grosses erreurs comptent beaucoup plus (on les met au carré).
- **Pourquoi ?** Pour voir si le modèle fait parfois de très grosses erreurs.
- **Ici : 126 924 $** sur le test final, bien plus que le MAE.
- **À retenir :** « RMSE bien plus grand que MAE = il y a quelques grosses erreurs. »

16. R^2

- **C'est quoi ?** La part de la variation des prix que le modèle explique. 1 = parfait, 0 = pas mieux que donner le prix moyen à tout le monde.
- **Pourquoi ?** Une note globale facile à comparer entre modèles.
- **Ici : 0,877** sur le test final.
- **À retenir :** « Le modèle explique environ 87,7 % de la variation des prix dans le jeu de test. » (et **pas** « 87,7 % des prix sont corrects »)

17. Surapprentissage (overfitting)

- **C'est quoi ?** Le modèle apprend par cœur au lieu de comprendre la tendance.
- **Pourquoi c'est un problème ?** Il est excellent sur les maisons apprises, mais mauvais sur des nouvelles maisons.
- **Ici :** arbre de décision 0,999 sur les maisons apprises contre 0,740 en validation croisée. La Random Forest réduit le problème (0,982 contre 0,855).
- **À retenir :** « On le repère en comparant le score sur les données apprises et sur des données jamais vues. »

18. Grid Search

- **C'est quoi ?** Une méthode qui teste automatiquement plusieurs réglages du modèle et garde le meilleur.
- **Pourquoi ?** Pour essayer d'améliorer la Random Forest sans tester à la main.
- **Ici :** 12 combinaisons × 3 parts de validation croisée = 36 entraînements. Meilleur réglage : 200 arbres, profondeur max 15, `min_samples_split` = 2.
- **À retenir :** « Grid Search n'est pas un modèle, c'est une façon de régler un modèle. »

19. Modèle final

- **C'est quoi ?** Le modèle qu'on garde et qu'on met dans l'application.
- **Pourquoi ce modèle ?** Meilleur R^2 en validation croisée (0,856).
- **Ici :** Random Forest optimisée, réentraînée sur tout le train, puis évaluée **une seule fois** sur le test : MAE 71 718 $, RMSE 126 924 $, R^2 0,877.

- **À retenir :** « Le score sur le test est proche de la validation croisée : pas de mauvaise surprise sur des maisons nouvelles. »

20. Sauvegarde avec joblib

- **C'est quoi ?** Écrire le modèle entraîné dans un fichier : `model.pkl`.
- **Pourquoi ?** L'application n'a pas à réentraîner le modèle à chaque lancement.
- **Ici :** `joblib.dump(modele_final, "model.pkl", compress=3)` → 26,3 Mo. On vérifie que le modèle rechargé donne la même prédiction.
- **À retenir :** « `model.pkl`, c'est le cerveau déjà entraîné qu'on donne à l'application. »

---

6. LES 3 MODÈLES EN UNE IMAGE MENTALE

	Régression linéaire	Arbre de décision	Random Forest
Image mentale	« une formule »	« une suite de questions »	« beaucoup d'arbres dont on fait la moyenne »
Idée simple	prix ≈ a × surface + b × grade + …	questions oui/non → groupe de maisons → prix moyen du groupe	chaque arbre voit un tirage différent des maisons, on fait la moyenne
Avantage	simple, rapide, facile à lire	facile à comprendre, voit les relations non linéaires	précise, limite le surapprentissage
Défaut	seulement des relations en ligne droite	apprend par cœur s'il n'est pas limité	plus lente, fichier plus lourd, moins facile à lire en détail
Résultat (validation croisée)	R^2 0,652 – MAE 138 007 $	R^2 0,740 – MAE 102 744 $	R^2 0,855 – MAE 74 266 $ (0,856 – 73 764 $ après Grid Search)
Décision	rejetée : trop simple	rejetée : surapprentissage	**gardée**

Attention : pour une **régression**, les arbres de la forêt ne « votent » pas, ils donnent chacun un prix et on fait la **moyenne**. (Le vote, c'est pour la classification.)

Pourquoi la Random Forest est meilleure ici ?

« Les relations entre les caractéristiques d'une maison et son prix ne sont pas toutes linéaires, par exemple l'effet de l'emplacement. La Random Forest peut mieux représenter ces relations, et elle réduit les faiblesses d'un arbre unique en faisant la moyenne de beaucoup d'arbres. »

---

7. LES MÉTRIQUES EN LANGAGE HUMAIN

Métrique	Version simple	Dans notre projet (test final)
MAE	erreur moyenne, en dollars	≈ 71 718 $
RMSE	comme le MAE, mais les grosses erreurs pèsent plus	≈ 126 924 $
R^2	part de la variation des prix expliquée par le modèle	≈ 0,877

Pour MAE et RMSE : plus c'est petit, mieux c'est. **Pour le R^2 :** plus c'est proche de 1, mieux c'est.

R^2 = 0,877

- ❌ NE PAS dire : « 87,7 % des prix sont corrects. »
- ❌ NE PAS dire : « Le modèle a 87,7 % de précision. »
- ✅ DIRE : « Le modèle explique environ 87,7 % de la variation des prix dans le jeu de test. »

Pour donner un ordre de grandeur : 71 718 $ d'erreur moyenne, c'est environ 16 % du prix médian (450 000 $).

Pourquoi deux séries de chiffres ?

- **Validation croisée** (sur le train) : sert à **choisir** le modèle → Random Forest optimisée, R^2 0,856.
- **Test final** (une seule fois) : sert à **vérifier** le modèle choisi → R^2 0,877.
- Le test est même un peu meilleur, car le modèle final apprend sur tout le train, alors qu'en validation croisée chaque modèle n'apprenait que sur 2/3 du train.

---

8. GRID SEARCH EXPLIQUÉ SIMPLEMENT

```
Random Forest
     ↓
12 combinaisons de réglages
     ↓
validation croisée (3 parts) sur le train pour chacune → 36 entraînements
     ↓
meilleure combinaison : 200 arbres, profondeur 15, min_samples_split 2
     ↓
modèle final
```

Grid Search n'est PAS un algorithme de prédiction. C'est une méthode pour tester automatiquement plusieurs réglages d'un modèle.

Paramètre	En une phrase	Valeurs testées	Meilleure valeur
`n_estimators`	le nombre d'arbres dans la forêt	100, 200	**200**
`max_depth`	le nombre maximum de questions à la suite dans chaque arbre (None = sans limite)	10, 15, None	**15**
`min_samples_split`	le nombre minimum de maisons dans un groupe pour pouvoir encore le couper en deux	2, 10	**2**

Résultat : gain très petit (R^2 0,855 → 0,856, MAE 74 266 $ → 73 764 $, environ 500 $ de mieux). Les réglages par défaut étaient déjà bons. La grille est volontairement petite pour que ça tourne en quelques minutes.

Mot technique : ces réglages s'appellent des **hyperparamètres** (on les choisit avant l'entraînement, le modèle ne les apprend pas tout seul).

---

9. POURQUOI SEULEMENT 8 VARIABLES ?

1. On ne voulait pas garder toutes les colonnes inutilement (modèle plus simple, formulaire plus court).
2. **Méthode 1 – corrélation :** première idée des variables liées au prix.
3. **Méthode 2 – sélection forward :** garde les variables qui apportent vraiment quelque chose en plus des autres.
4. **Vérification :** Random Forest en validation croisée → R^2 **0,856 avec les 15 variables**, **0,855 avec les 8**. Différence presque nulle.

Variable	En français	Importance dans le modèle final
`sqft_living`	surface habitable (pieds² ; 1 m² ≈ 10,76 pieds²)	0,319
`grade`	qualité de construction, de 1 à 13 (7 = moyen)	0,311
`lat`	latitude : position nord-sud	0,188
`long`	longitude : position est-ouest	0,093
`waterfront`	vue sur l'eau (0 = non, 1 = oui)	0,041
`sqft_lot`	surface du terrain	0,038
`bedrooms`	nombre de chambres	0,006
`yr_renovated`	année de rénovation (0 = jamais rénovée)	0,003

À retenir : surface + qualité ≈ 2/3 de l'importance, puis l'emplacement (`lat` + `long` ≈ 0,28). `bedrooms` et `yr_renovated` servent très peu, mais ont été choisies par la sélection.

Variables non gardées qu'on peut vous demander : `bathrooms` (déjà dans la surface, corrélation 0,76 avec `sqft_living`), `sqft_living15` (pareil), `view` (en partie dans `waterfront` et la position).

---

10. FLASK ≠ DOCKER

FLASK	DOCKER
une librairie Python pour faire une application web	un outil pour emballer une application dans un « conteneur »
affiche le formulaire	contient l'application (`app.py` , `model.pkl` , HTML, CSS)
reçoit les valeurs envoyées	contient Python 3.12
appelle le modèle (`model.predict`)	contient les librairies (`requirements.txt`)
affiche le prix estimé	permet de lancer le projet de la même manière partout

Phrase clé :

> « Flask fait fonctionner l'application. Docker transporte et exécute l'environnement complet de l'application. »

- **Sans Docker :** il faut installer Python et les bonnes versions des librairies sur chaque machine.
- **Avec Docker :** l'environnement est déjà décrit dans le `Dockerfile` , il suffit d'avoir Docker.

Le Dockerfile ligne par ligne :

Instruction	Ce qu'elle fait
`FROM python:3.12-slim`	part d'un système léger avec Python 3.12
`WORKDIR /app`	se place dans le dossier `/app` du conteneur
`COPY requirements.txt .`	copie la liste des librairies
`RUN pip install --no-cache-dir -r requirements.txt`	installe les librairies
`COPY app.py model.pkl ./` + `templates` + `static`	copie l'application
`EXPOSE 5000`	indique que l'application utilise le port 5000
`CMD ["python", "app.py"]`	commande lancée au démarrage du conteneur

Librairies de l'application (`requirements.txt`) : flask 3.0.0, scikit-learn 1.8.0, pandas 2.3.3, joblib 1.5.3 (numpy s'installe automatiquement avec). La version de scikit-learn doit être la même que celle qui a créé `model.pkl`.

Détail utile : dans `app.py`, `app.run(host="0.0.0.0", port=5000)`. Le `0.0.0.0` permet d'accéder à l'application depuis l'extérieur du conteneur.

---

11. LANCER LE PROJET (fiche de secours)

Toutes les commandes se lancent à la racine du projet : `D:\Laplateforme\B3 IA\Apartment-Hunter` (dépôt GitHub : `RomainJazzar/Apartment-Hunter`).

Flask en local (sans Docker)

Première fois seulement (créer l'environnement virtuel et installer les librairies — déjà fait chez nous, le dossier `.venv` existe) :

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Ensuite, à chaque fois :

```
.venv\Scripts\activate
python app.py
```

Puis ouvrir `http://localhost:5000`

- `python -m venv .venv` → crée un environnement Python isolé pour le projet.
- `.venv\Scripts\activate` → active cet environnement (sur Windows).
- `pip install -r requirements.txt` → installe les librairies listées.
- `python app.py` → lance l'application Flask sur le port 5000.

Si PowerShell refuse d'activer l'environnement, on peut lancer directement `.venv\Scripts\python app.py`.

Docker

```
docker build -t apartment-hunter .
docker run --rm -p 5000:5000 apartment-hunter
```

Puis ouvrir `http://localhost:5000` (Docker Desktop doit être ouvert).

- `docker build -t apartment-hunter .` → construit l'image à partir du `Dockerfile` et l'appelle `apartment-hunter`.
- `docker run --rm -p 5000:5000 apartment-hunter` → lance un conteneur ; `-p 5000:5000` relie le port 5000 de l'ordinateur au port 5000 du conteneur ; `--rm` supprime le conteneur quand on l'arrête.

Notebook

Le notebook a été conçu pour **Google Colab** : créer un dossier `data` dans l'onglet *Fichiers*, y déposer `kc_house_data.csv`, puis *Exécution* → *Tout exécuter*.

Dans VS Code (optionnel) :

- activer `.venv` et installer en plus `pip install matplotlib ipykernel` (le notebook a besoin de matplotlib, pas l'application)
- ouvrir `apartment_hunter.ipynb`
- choisir le kernel `.venv`
- *Run All*

Le notebook complet tourne en quelques minutes ; la partie la plus longue est le Grid Search.

---

12. RÉPONSES DE SECOURS (si le prof me coupe)

Données et nettoyage

1. **Pourquoi King County ?** → Moins de colonnes, pas de texte libre et des variables bien documentées : plus simple à nettoyer et à expliquer.
2. **Combien de données ?** → 21 613 ventes et 21 colonnes au départ, 21 596 maisons et 16 colonnes après nettoyage.
3. **Il y avait des valeurs manquantes ?** → Non, aucune. On l'a vérifié avec `isna().sum()`.
4. **Et des doublons ?** → Aucune ligne en double. 177 maisons vendues deux fois, à des dates différentes : on les a gardées.
5. **Pourquoi supprimer `id` ?** → C'est un identifiant, il n'explique pas le prix.
6. **Pourquoi supprimer `date` ?** → Les ventes couvrent seulement environ 13 mois, et on a choisi de ne pas étudier l'effet du temps pour garder le projet simple.
7. **Pourquoi pas le code postal ?** → C'est un code, pas une quantité ; `lat` et `long` donnent déjà la position avec de vrais nombres.
8. **Pourquoi enlever `sqft_above` et `sqft_basement` ?** → Leur somme est exactement `sqft_living` : c'était de l'information en double.
9. **Pourquoi garder les maisons très chères ?** → Ce ne sont pas des erreurs mais des maisons de luxe ; les enlever empêcherait le modèle d'estimer les grandes maisons.

10. **Qu'est-ce que vous avez supprimé comme lignes ?** → 17 lignes impossibles : une maison à 33 chambres pour 150 m² et 16 maisons sans chambre ou sans salle de bain.
11. **Pourquoi pas de normalisation ?** → Les arbres n'en ont pas besoin, et ça ne change pas les prédictions de la régression linéaire.
12. **Pourquoi pas Power BI ?** → On n'y avait pas accès ; on a fait l'analyse en Python et un `dashboard.html` consultable.

Variables 13. **Pourquoi 8 variables ?** → Même performance qu'avec les 15 (0,855 contre 0,856), avec un modèle plus simple et un formulaire plus court. 14. **Pourquoi deux méthodes de sélection ?** → La corrélation ne voit que les liens en ligne droite ; la sélection forward regarde ce que chaque variable apporte en plus des autres. 15. **Pourquoi garder la longitude alors que sa corrélation est 0,02 ?** → L'emplacement compte beaucoup mais pas en ligne droite ; la sélection forward l'a gardée et c'est le 4e facteur du modèle. 16. **Pourquoi pas `bathrooms` ?** → Elle est très liée à la surface (0,76) : une fois la surface connue, elle n'apporte presque rien. 17. **Pourquoi la sélection seulement sur le train ?** → Pour que le test reste complètement nouveau et que le score final soit honnête. 18. **Pourquoi pas Boruta ?** → C'était un exemple du sujet ; deux méthodes plus simples suffisaient.

Modèles et métriques 19. **Pourquoi Random Forest ?** → Elle donnait les meilleurs résultats en validation croisée et représente mieux les relations complexes qu'une régression linéaire. 20. **Pourquoi la régression linéaire est moins bonne ?** → Elle ne fait que des lignes droites, alors que l'effet de l'emplacement n'est pas linéaire. 21. **C'est quoi le surapprentissage ?** → Le modèle apprend par cœur : l'arbre seul fait 0,999 sur les maisons apprises mais 0,740 sur les autres. 22. **Pourquoi train / test ?** → Pour vérifier le modèle sur des maisons qu'il n'a jamais vues. 23. **Pourquoi la validation croisée ?** → Pour comparer et régler les modèles sans utiliser le jeu de test. 24. **Que veut dire $R^2 = 0{,}877$?** → Le modèle explique environ 87,7 % de la variation des prix dans le jeu de test. 25. **Que veut dire MAE = 71 718 $?** → En moyenne, l'estimation s'écarte d'environ 72 000 $ du vrai prix. 26. **Pourquoi le RMSE est plus grand que le MAE ?** → Le RMSE donne plus de poids aux grosses erreurs : il y en a quelques-unes. 27. **72 000 $ d'erreur, c'est beaucoup ?** → C'est environ 16 % du prix médian : correct pour une première estimation, pas un prix exact. 28. **Pourquoi le test est meilleur que la validation croisée ?** → Le modèle final apprend sur tout le train, alors qu'en validation croisée chaque modèle n'apprenait que sur 2/3. 29. **Comment avez-vous choisi le modèle sans tricher ?** → Tous les choix par validation croisée sur le train ; le test n'a servi qu'une fois, à la fin.

Grid Search et modèle final 30. **C'est quoi Grid Search ?** → Une méthode qui teste automatiquement plusieurs réglages du modèle et garde le meilleur. 31. **Pourquoi si peu de gain ?** → Les réglages par défaut étaient déjà bons ; on gagne environ 500 $ d'erreur moyenne. 32. **Pourquoi une si petite grille ?** → 12 combinaisons suffisent pour montrer la méthode, et ça tourne en quelques minutes. 33. **Quels sont les réglages finaux ?** → 200 arbres, profondeur maximale de 15, `min_samples_split` = 2.

Application et Docker 34. **Pourquoi `model.pkl` ?** → Pour sauvegarder le modèle déjà entraîné et éviter de le réentraîner à chaque lancement. 35. **Pourquoi joblib ?** → C'est l'outil présenté dans la documentation de scikit-learn pour enregistrer et recharger un modèle, et il gère bien les gros modèles. 36. **Pourquoi `compress=3` ?** → Pour réduire le fichier à 26,3 Mo et rester sous la limite de 100 Mo de GitHub. 37. **Pourquoi Flask ?** → Pour rendre le modèle utilisable depuis une interface web. 38. **Le modèle est chargé à chaque clic ?** → Non, une seule fois au démarrage de l'application ; à chaque clic on appelle seulement `predict`. 39. **Pourquoi un DataFrame dans `app.py` ?** → Le modèle a appris avec un tableau pandas : on lui redonne les 8 colonnes, avec les mêmes noms et dans le même ordre. 40. **Pourquoi Docker ?** → Pour fournir la même application avec le même environnement et les mêmes dépendances sur différentes machines. 41. **Flask et Docker, c'est pareil ?** → Non : Flask fait fonctionner l'application, Docker emballe tout l'environnement qui la fait tourner. 42. **Pourquoi le port 5000 ?** → C'est le port par défaut de Flask ; on le relie à l'ordinateur avec `-p 5000:5000`. 43. **C'est une API ?** → Non, c'est une simple page web avec un formulaire ; on n'a pas fait d'API. 44. **Le modèle marche pour Paris ?** → Non : il a appris sur des ventes de 2014-2015 dans la région de Seattle uniquement. 45. **Pourquoi demander la latitude et la longitude ?** → C'est l'emplacement, un des facteurs les plus importants du prix, sous forme de nombres simples.

---

13. DICTIONNAIRE DES MOTS TECHNIQUES

Mot	Définition en une phrase
dataset	le jeu de données : ici le tableau des 21 613 ventes de maisons.
feature (variable explicative)	une information donnée au modèle pour prédire, comme la surface ou le grade.
target (cible)	ce qu'on veut prédire : ici `price`.
régression	prédire un nombre (un prix), pas une catégorie.
entraînement	la phase où le modèle apprend à partir des maisons dont on connaît le prix.
prédiction	le prix que le modèle propose pour une maison.
validation	mesurer un modèle sur des données mises de côté pour comparer ou régler des modèles.
test	les 20 % de maisons gardées pour évaluer le modèle final une seule fois, à la fin.
overfitting (surapprentissage)	quand le modèle apprend les données par cœur et se trompe sur des données nouvelles.
hyperparamètre	un réglage du modèle choisi avant l'entraînement, comme le nombre d'arbres.
Grid Search	une méthode qui teste toutes les combinaisons d'hyperparamètres d'une grille et garde la meilleure.
cross-validation (validation croisée)	couper le train en plusieurs parts et tester chaque part à tour de rôle pour avoir un score fiable.
MAE	l'erreur moyenne en dollars entre le prix estimé et le vrai prix.
RMSE	une erreur en dollars qui pénalise davantage les grosses erreurs.
R^2	la part de la variation des prix expliquée par le modèle (1 = parfait).
Random Forest	un modèle qui fait la moyenne de beaucoup d'arbres de décision un peu différents.
Flask	une librairie Python pour créer une petite application web.
Docker	un outil qui emballe une application et ses dépendances dans un conteneur qui marche partout pareil.
joblib	une librairie pour enregistrer un modèle dans un fichier et le recharger.
model.pkl	le fichier qui contient notre Random Forest déjà entraînée.
DataFrame	un tableau de données pandas, avec des lignes et des colonnes nommées.
API	un programme qui répond à d'autres programmes (pas à un humain) : notre projet **n'en a pas**, c'est une page web avec un formulaire.
port 5000	la « porte » réseau sur laquelle l'application Flask écoute : on l'ouvre avec `http://localhost:5000`.

---

14. CE QU'IL FAUT COMPRENDRE, PAS RÉCITER

L'idée : chacun présente sa partie, mais **tout le monde** doit pouvoir répondre aux questions de base sur l'ensemble (sections 2 et 12).

Romain → données, nettoyage, analyse exploratoire, logique globale

À comprendre :

- d'où viennent les données et ce que représente une ligne (une vente de maison)
- la différence entre une valeur **impossible** (à supprimer) et une valeur **extrême** (à garder)
- pourquoi chaque colonne supprimée l'a été
- pourquoi la corrélation de la longitude est faible alors que la carte montre que l'emplacement compte
- le chemin complet : CSV → notebook → `model.pkl` → Flask → Docker (section 1)

Yannis → sélection des variables, modèles, métriques

À comprendre :

- la différence entre corrélation et sélection forward
- pourquoi 8 variables suffisent (0,855 contre 0,856)
- l'idée de chaque modèle : formule, questions, moyenne d'arbres
- ce que mesurent MAE, RMSE et R^2, et la bonne phrase pour le R^2
- le surapprentissage (0,999 contre 0,740)
- pourquoi on compare en validation croisée et pas sur le test

Lisa → Grid Search, modèle final, Flask, Docker

À comprendre :

- Grid Search = méthode de réglage, pas un modèle ; les 3 paramètres et leur sens
- pourquoi le gain est petit, et pourquoi c'est normal
- pourquoi le test n'est utilisé qu'une fois, et pourquoi il est un peu meilleur que la validation croisée
- le trajet d'une estimation (section 4), et le fait que le modèle est chargé une seule fois
- la différence Flask / Docker, et ce que fait chaque ligne du Dockerfile

---

15. LE PROJET EN 60 SECONDES

« Notre projet, Apartment Hunter, répond à une question simple : peut-on estimer le prix d'une maison à partir de ses caractéristiques ? On a utilisé environ 21 600 ventes de maisons de la région de Seattle. Les données étaient propres : on a seulement enlevé 17 lignes impossibles et quelques colonnes inutiles, en gardant les maisons de luxe. Les graphiques montrent que le prix dépend surtout de la surface, de la qualité de construction et de l'emplacement. On a gardé 8 variables avec deux méthodes de sélection, sans perdre de précision. On a comparé trois modèles en validation croisée : régression linéaire, arbre de décision et Random Forest. La Random Forest est la meilleure, et on l'a réglée avec un Grid Search. Sur des maisons jamais vues, elle explique environ 87,7 % de la variation des prix, avec une erreur moyenne d'environ 72 000 dollars. Le modèle est sauvegardé dans un fichier, utilisé par une application Flask où l'on remplit un formulaire, et le tout est emballé avec Docker pour fonctionner partout. »

16. LE PROJET EN 20 SECONDES

« On a entraîné une Random Forest sur 21 600 ventes de maisons pour estimer leur prix. Elle explique environ 88 % de la variation des prix, avec environ 72 000 dollars d'erreur moyenne. On l'utilise dans une application Flask, emballée avec Docker. »